

T-Sharp (T#) — Ağ İşlemleri

Başlamadan Önce — `kullan ag`

Ağ modülünü kullanabilmek için dosyanın **en başına** şu satırı eklemelisiniz:

```
kullan ag
```

Bu komut, `requests` kütüphanesini aktive eder ve `TSharp/4.1` User-Agent başlıklı bir oturum (session) açar. Tüm ağ komutları bu oturum üzerinden çalışır.

Gereksinim: `pip install requests`

İçindekiler

1. HTTP GET — `http_get`
2. HTTP POST — `http_post`
3. HTTP PUT — `http_put`
4. HTTP DELETE — `http_delete`
5. HTTP PATCH — `http_patch`
6. HTTP HEAD — `http_head`
7. JSON Al — `json_al`
8. JSON Gönder — `json_gonder`
9. Dosya İndir — `dosya_indir`
10. Stream İndir — `stream_indir`
11. Çoklu Dosya Gönder — `coklu_dosya_gonder`
12. Ping — `ping`
13. Oturum Ayarları
14. Durum Kodu Alma — `durum_kodu`
15. Tam Örnek Programlar

1. HTTP GET — `http_get`

Bir URL'den veri çeker.

Söz dizimi

```
http_get <degisken> "<url>"
```

Otomatik Oluşan Değişkenler

Değişken	İçerik
<degisken>	Ham response nesnesi
<degisken>_metin	Yanıtın metin içeriği
<degisken>_kod	HTTP durum kodu (200, 404 vb.)

Örnekler

```
kullan ag

// Basit GET isteği
http_get yanıt "https://httpbin.org/get"
yazdır yanıt_metin
```

```
kullan ag

// Durum kodu kontrolü
http_get yanıt "https://httpbin.org/status/200"

eger yanıt_kod esittir 200:
    yazdır "İstek başarılı!"
degilse:
    yazdır "Hata kodu:" yanıt_kod
son
```

2. HTTP POST — http_post

Bir URL'ye form verisi gönderir.

Söz dizimi

```
http_post <degisken> "<url>" <veri>
```

Örnekler

```
kullan ag

sozluk form_veri {"kullanici": "talha", "sifre": "1234"}

http_post yanit "https://httpbin.org/post" form_veri
yazdir yanit_metin
yazdir "Durum kodu:" yanit_kod
```

3. HTTP PUT — `http_put`

Sunucudaki bir kaynağı günceller (tam güncelleme).

Söz dizimi

```
http_put <degisken> "<url>" <veri>
```

Örnek

```
kullan ag

sozluk guncelleme {"baslik": "Yeni Başlık", "icerik": "Güncel içerik"}

http_put yanit "https://httpbin.org/put" guncelleme
yazdir yanit_kod
```

4. HTTP DELETE — `http_delete`

Sunucudaki bir kaynağı siler.

Söz dizimi

```
http_delete <degisken> "<url>"
```

Örnek

```
kullan ag

http_delete yanit "https://httpbin.org/delete"

eger yanit_kod esittir 200:
    yazdir "Kayıt başarıyla silindi."
son
```

5. HTTP PATCH — `http_patch`

Sunucudaki bir kaynağı kısmen günceller.

Söz dizimi

```
http_patch <degisken> "<url>" <veri>
```

Örnek

```
kullan ag

sozluk degisiklik {"durum": "aktif"}

http_patch yanit "https://httpbin.org/patch" degisiklik
yazdir yanit_kod
```

6. HTTP HEAD — `http_head`

Yanıt gövdesini almadan yalnızca başlıkları kontrol eder. Dosya boyutu veya içerik türü öğrenmek için kullanışlıdır.

Söz dizimi

```
http_head <degisken> "<url>"
```

Örnek

```
kullan ag

http_head yanit "https://httpbin.org/get"
yazdir "Sunucu durumu:" yanit_kod
```

7. JSON AI — json_al

Bir API'den doğrudan JSON verisi çeker ve otomatik olarak sözlüğe/listeye dönüştürür.

Söz dizimi

```
json_al <degisken> "<url>"
```

Örnekler

```
kullan ag

// Hava durumu API örneği (açık API)
json_al veri "https://jsonplaceholder.typicode.com/todos/1"

yazdir veri["title"]
yazdir veri["completed"]
```

```
kullan ag

// Liste döndüren API
json_al kullanicilar "https://jsonplaceholder.typicode.com/users"

// İlk kullanıcıyı yazdır
degisken ilk_kullanici = kullanicilar[0]
degisken ilk_isim      = ilk_kullanici["name"]
degisken ilk_email     = ilk_kullanici["email"]
yazdır ilk_isim
yazdır ilk_email

// Tüm isimleri listele
her kullanıcı icinde kullanicilar:
    degisken kadi = kullanıcı["name"]
    yazdır kadi
son
```

8. JSON Gönder — json_gonder

Bir sözlüğü JSON formatında POST eder.

Söz dizimi

```
json_gonder <degisken> "<url>" <sozluk_verisi>
```

Örnek

```
kullan ag

sozluk yeni_kayit {
  "title": "T-Sharp ile yazıldı",
  "body": "Bu gönderi T# kullanılarak oluşturuldu.",
  "userId": 1
}

json_gonder yanit "https://jsonplaceholder.typicode.com/posts" yeni_kayit

yazdir "Durum:" yanit_kod
```

9. Dosya İndir — `dosya_indir`

İnternet üzerinden bir dosyayı indirip diske kaydeder. İlerleme yüzdesi terminalde gösterilir. Maksimum 500 MB desteklenir.

Söz dizimi

```
dosya_indir "<url>" "<kayit_yolu>"
```

Örnekler

```
kullan ag

// Resim indir
dosya_indir "https://upload.wikimedia.org/wikipedia/commons/4/47/PNG_transparency_demonstration_1.png" "resim.png"
yazdir "İndirme tamamlandı!"
```

```
kullan ag

// Alt klasöre indir (klasör yoksa otomatik oluşturulur)
dosya_indir "https://example.com/rapor.pdf" "indirilenler/rapor.pdf"
```

10. Stream İndir — `stream_indir`

Büyük dosyaları parça parça (chunk) indirmek için kullanılır. Bellek dostu bir yöntemdir.

Söz dizimi

```
stream_indir "<url>" "<kayit_yolu>"
```

Örnek

```
kullan ag
```

```
// Büyük dosya için stream indirme  
stream_indir "https://example.com/buyuk_video.mp4" "video.mp4"
```

`dosya_indir` vs `stream_indir`: `dosya_indir` ilerleme yüzdesi gösterir ve boyut sınırı uygular. `stream_indir` ise daha ham ve hafızayı daha az kullanan bir yöntemdir — büyük medya dosyaları için tercih edin.

11. Çoklu Dosya Gönder — `coklu_dosya_gonder`

Birden fazla dosyayı tek bir POST isteğiyle gönderir.

Söz dizimi

```
coklu_dosya_gonder <degisken> "<url>" <dosya_listesi>
```

Örnek

```
kullan ag
```

```
liste yuklenecekler ["resim1.png", "rapor.pdf", "veri.csv"]
```

```
coklu_dosya_gonder yanit "https://example.com/yukle" yuklenecekler  
yazdir "Yükleme durumu:" yanit_kod
```

Listede bulunamayan dosyalar atlanır ve uyarı verilir.

12. Ping — ping

Bir sunucuya istek gönderip yanıt süresini milisaniye cinsinden ölçer.

Söz dizimi

```
ping <degisken> "<url>"
```

Otomatik Oluşan Değişkenler

Değişken	İçerik
<degisken>	Yanıt süresi (ms), hata durumunda -1
<degisken>_durum	HTTP durum kodu

Örnek

```
kullan ag

ping sure "https://www.google.com"

eger sure esittir -1:
    yazdır "Sunucuya ulaşamadı."
degilse:
    yazdır "Yanıt süresi:" yuvarla(sure, 1) "ms"
    yazdır "Durum kodu:" sure_durum

son
```

13. Oturum Ayarları

Tüm ağ isteklerine uygulanacak genel ayarları oturum başında yapılandırabilirsiniz.

Özel Başlık Ekle — baslik_ekle

```
baslik_ekle "<baslik_adi>" "<deger>"
```

```
kullan ag
```

```
baslik_ekle "Authorization" "Bearer gizli_token_buraya"
```

```
baslik_ekle "Accept-Language" "tr-TR"
```

```
http_get yanit "https://api.example.com/veri"
```

Çerez Ekle — `cerez_ekle`

```
cerez_ekle "<cerez_adi>" "<deger>"
```

```
kullan ag
```

```
cerez_ekle "oturum_id" "abc123xyz"
```

```
http_get yanit "https://example.com/profil"
```

Zaman Aşımı Ayarla — `timeout_ayarla`

Varsayılan zaman aşımı 30 saniyedir. İstediğiniz gibi değiştirebilirsiniz.

```
timeout_ayarla <saniye>
```

```
kullan ag
```

```
// Yavaş API için 60 saniye bekle
```

```
timeout_ayarla 60
```

```
http_get yanit "https://yavas-api.example.com/veri"
```

Proxy Ayarla — `proxy_ayarla`

```
proxy_ayarla "<proxy_url>"
```

```
kullan ag
```

```
proxy_ayarla "http://proxy.sirket.com:8080"
```

```
http_get yanit "https://example.com"
```

14. Durum Kodu Alma — durum_kodu

Bir response nesnesinin HTTP durum kodunu ayrı bir değişkene atar.

```
durum_kodu <response_degiskeni> <kod_degiskeni>
```

kullan ag

```
http_get yanit "https://httpbin.org/get"
```

```
durum_kodu yanit istek_kodu
```

```
yazdır istek_kodu
```

Not: `http_get`, `http_post` vb. komutlar `<degisken>_kod` değişkenini zaten otomatik oluşturur. `durum_kodu` komutu, response nesnesini farklı bir değişkende tuttuğunuzda kullanışlıdır.

15. Tam Örnek Programlar

Örnek 1 — Açık Hava Durumu API'si

```
kullan ag

// Open-Meteo: ücretsiz, kayıt gerektirmeyen hava durumu API'si
degisken url = "https://api.open-meteo.com/v1/forecast?latitude=39.93&longitude=32.86
&current_weather=true"

json_al hava url

eger hava degildir hic:
    degisken simdiki    = hava["current_weather"]
    degisken sicaklik   = simdiki["temperature"]
    degisken ruzgar     = simdiki["windspeed"]
    degisken hava_kodu  = simdiki["weathercode"]

    yazdır "=== ANKARA HAVA DURUMU ==="
    yazdır "Sıcaklık    :" sicaklik "°C"
    yazdır "Rüzgar      :" ruzgar  "km/s"
    yazdır "Hava kodu   :" hava_kodu

degilse:
    yazdır "Hava durumu alınamadı."

son
```

Örnek 2 — REST API CRUD İşlemleri

kullan ag

```
degisken base_url = "https://jsonplaceholder.typicode.com/posts"
```

```
// --- OKUMA (GET) ---
```

```
yazdır ">>> Gönderi listesi alınıyor..."
```

```
json_al gonderiler base_url
```

```
yazdır "Toplam gönderi:" uzunluk(gonderiler)
```

```
degisken ilk_baslik = gonderiler[0]["title"]
```

```
yazdır "İlk gönderi:" ilk_baslik
```

```
yazdır
```

```
// --- OLUŞTURMA (POST) ---
```

```
yazdır ">>> Yeni gönderi oluşturuluyor..."
```

```
sozluk yeni_gonderi {
```

```
    "title": "T-Sharp ile REST",
```

```
    "body": "Bu gönderi T# v4.1 ile oluşturuldu.",
```

```
    "userId": 1
```

```
}
```

```
json_gonder olustur_yanit base_url yeni_gonderi
```

```
yazdır "Oluşturma durumu:" olustur_yanit_kod
```

```
yazdır
```

```
// --- GÜNCELLEME (PUT) ---
```

```
yazdır ">>> Gönderi güncelleniyor..."
```

```
sozluk guncelleme {
```

```
    "title": "Güncellenmiş Başlık",
```

```
    "body": "Güncellenmiş içerik.",
```

```
    "userId": 1
```

```
}
```

```
http_put guncelle_yanit base_url + "/1" guncelleme
```

```
yazdır "Güncelleme durumu:" guncelle_yanit_kod
```

```
yazdır
```

```
// --- SİLME (DELETE) ---
```

```
yazdır ">>> Gönderi siliniyor..."
```

```
http_delete sil_yanit base_url + "/1"
```

```
yazdır "Silme durumu:" sil_yanit_kod
```

Örnek 3 — Toplu Dosya İndirici

Not: İç içe liste (`[["url","hedef"], ...]`) söz dizimi desteklenmez. URL ve hedef listeleri **ayrı ayrı** tanımlanmalı, indeks ile eşleştirilmelidir.

kullan ag

```
// URL ve hedef yolları ayrı listeler olarak tanımla
degisken url1   = "https://httpbin.org/image/png"
degisken hedef1 = "resimler/ornek.png"
degisken url2   = "https://httpbin.org/robots.txt"
degisken hedef2 = "metin/robots.txt"
degisken url3   = "https://httpbin.org/json"
degisken hedef3 = "metin/ornek.json"

liste url_listesi   [url1, url2, url3]
liste hedef_listesi [hedef1, hedef2, hedef3]

degisken basarili   = 0
degisken basarisiz  = 0
degisken i          = 0

dongu i kucuktur uzunluk(url_listesi):
    degisken url   = url_listesi[i]
    degisken hedef = hedef_listesi[i]

    // Hedef klasörü oluştur (varsa hata vermez)
    degisken klasor_yolu = dilim(hedef, 0, bul(hedef, "/"))
    eger uzunluk(klasor_yolu) buyuktur 0:
        klasor_olustur(klasor_yolu)
    son

    yazdır "İndiriliyor:" url
    dosya_indir url hedef

    eger dosya_var_mi(hedef):
        basarili += 1
        yazdır "Tamamlandı:" hedef
    degilse:
        basarisiz += 1
        yazdır "Basarisiz:" hedef
    son
    yazdır
    i += 1
son

yazdır "=== ÖZET ==="
```

```
yazdır "Basarili :" basarili
yazdır "Basarisiz:" basarisiz
```

Örnek 4 — Sunucu Sağlık Kontrolü

```
kullan ag

liste sunucular [
    "https://www.google.com",
    "https://www.github.com",
    "https://httpbin.org",
    "https://var-olmayan-site-12345.com"
]

yazdır "=== SUNUCU SAĞLIK KONTROLÜ ==="
yazdır

her sunucu icinde sunucular:
    ping sure sunucu

    eger sure esittir -1:
        yazdır "X" sunucu "→ ERIŞİLEMİYOR"
    degilse:
        eger sure kucuktur 300:
            degisken durum = "HIZLI"
        degilse:
            eger sure kucuktur 1000:
                degisken durum = "NORMAL"
            degilse:
                degisken durum = "YAVAŞ"
        son
    son
    yazdır "✓" sunucu "→" yuvarla(sure, 0) "ms |" durum
son
son
```


Örnek 5 — API Token ile Kimlik Doğrulama İstek

```
kullan ag

// Token'ı güvenli şekilde kullanıcıdan al
girdi token "API Token'ınızı girin: "

// Oturum başlığına ekle (tüm isteklere uygulanır)
baslik_ekle "Authorization" "Bearer " + token
baslik_ekle "Content-Type" "application/json"

// Zaman aşımını ayarla
timeout_ayarla 15

// İsteği gönder
http_get yanıt "https://httpbin.org/bearer"

eger yanıt_kod esittir 200:
    yazdır "Kimlik doğrulama başarılı!"
    yazdır yanıt_metin
degilse:
    eger yanıt_kod esittir 401:
        yazdır "Hata: Geçersiz token."
    degilse:
        yazdır "Beklenmeyen hata, kod:" yanıt_kod
    son
son
```

Hızlı Başvuru Tablosu

Komut	Açıklama
<code>kullan ag</code>	Ağ modülünü aktive eder (zorunlu)
<code>http_get <d> "<url>"</code>	GET isteği
<code>http_post <d> "<url>" <veri></code>	POST isteği (form verisi)
<code>http_put <d> "<url>" <veri></code>	PUT isteği
<code>http_delete <d> "<url>"</code>	DELETE isteği
<code>http_patch <d> "<url>" <veri></code>	PATCH isteği
<code>http_head <d> "<url>"</code>	HEAD isteği
<code>json_al <d> "<url>"</code>	JSON GET → sözlüğe dönüştürür
<code>json_gonder <d> "<url>" <sozluk></code>	JSON POST
<code>dosya_indir "<url>" "<yol>"</code>	Dosya indirir (maks 500 MB)
<code>stream_indir "<url>" "<yol>"</code>	Büyük dosya stream indirir
<code>coklu_dosya_gonder <d> "<url>" <liste></code>	Çoklu dosya yükler
<code>ping <d> "<url>"</code>	Yanıt süresini ölçer (ms)
<code>baslik_ekle "<ad>" "<deger>"</code>	Oturum başlığı ekler
<code>cerez_ekle "<ad>" "<deger>"</code>	Çerez ekler
<code>timeout_ayarla <saniye></code>	Zaman aşımı ayarlar
<code>proxy_ayarla "<url>"</code>	Proxy ayarlar
<code>durum_kodu <response> <degisken></code>	Durum kodunu ayrı alır

Otomatik Değişkenler (GET/POST/vb. sonrası)

Değişken	İçerik
<code><d></code>	Ham response nesnesi
<code><d>_metin</code>	Yanıt metni (string)
<code><d>_kod</code>	HTTP durum kodu
<code>ping_d</code>	Süre (ms), <code>-1</code> = erişilemiyor
<code>ping_d_durum</code>	HTTP durum kodu

Sonraki bölüm: `04_SIFRELE_VE_HASH.md` — Base64, XOR şifreleme, SHA256/512, HMAC ve dosya şifreleme.